# Mathematical Background for `AccumulatR`

Niek Stevenson

**Abstract**

`AccumulatR` is built around one simple idea: each latent process has a random finishing time. Responses are defined by rules about which processes must finish, which must not finish, and which finish first. This note gives a compact mathematical description of that framework. The aim is not maximal generality. The aim is a notation that is simple enough to read from start to finish and precise enough to match the package.

## 1 Overview

The framework has four layers.

1. A *leaf* is a single latent finishing-time process.
2. A *pool* combines several sources into one new source.
3. An *outcome rule* says when a modeled response counts as having occurred.
4. An *observation layer* turns latent outcomes into the data we actually record, for example through mixtures, remapped labels, or ranked responses.

Everything is built from finishing times. Once an object has a finishing time, it can compete with other objects in exactly the same way.

## 2 Leaves and Basic Notation

A leaf is the basic latent process in the model. If leaf $a$ finishes at random time $T_a$, write

$$F_a(t) = \Pr(T_a \leq t), \qquad f_a(t) = \frac{d}{dt} F_a(t), \qquad S_a(t) = 1 - F_a(t).$$

So $F_a(t)$ is the probability that leaf $a$ has finished by time $t$, $f_a(t)$ is the density that it finishes at time $t$, and $S_a(t)$ is the probability that it is still unfinished at time $t$.

The same notation will be used for pools and outcome rules as well. Once an object has a finishing time, it is treated in exactly the same way.

In this note, a *source* means any object with its own finishing time. The two basic source types are leaves and pools.

Different leaves may have different finishing-time distributions. The package currently supplies the families

`lognormal`, `gamma`, `exgauss`, `LBA`, `RDM`.

But the mathematics below does not depend on a particular family. It only requires that each leaf has well-defined functions $F_a$, $f_a$, and $S_a$.

## 3 Pools

A pool turns several sources into one new source. Suppose pool $P$ has members with finishing times $T_1, \ldots, T_n$, and suppose the pool finishes when the $k$th member finishes. Then $T_P$ is the $k$th order statistic of $T_1, \ldots, T_n$.

The defining identity is

$$F_P(t) = \Pr(T_P \le t) = \Pr(\text{at least } k \text{ of } T_1, \ldots, T_n \text{ are } \le t). \tag{1}$$

That is the cleanest way to think about a pool: by time $t$, the pool has finished exactly when enough of its members have finished.

If the pool members are exchangeable, with common CDF $F$ and density $f$, then

$$F_P(t) = \sum_{m=k}^{n} \binom{n}{m} F(t)^m S(t)^{n-m}, \tag{2}$$

and

$$f_P(t) = \frac{n!}{(k-1)!(n-k)!}\, f(t)\, F(t)^{k-1} S(t)^{n-k}. \tag{3}$$

These are the usual formulas for the $k$th finisher.

The package also allows pools whose members have different distributions. The notation is then longer, but the idea is the same: a pool is simply another source with its own $F_P$, $f_P$, and $S_P$.

Three special cases matter often:

- $k = 1$: the pool is a first finisher.

- $k = n$: the pool is a last finisher.
- $n = 1$: the pool is just an alias for one source.

Pools can also be nested. Once a source has its own $F$, $f$, and $S$, it can be used as a member of a larger pool.

# 4 Outcome Rules

An outcome rule says when a modeled response becomes available. If $E$ is an outcome rule, then $T_E$ is the first time that rule becomes true.

## 4.1 Direct Outcomes

The simplest outcome rule is a direct readout from one source. In that case, the outcome simply inherits the finishing-time distribution of that source. Nothing new is added mathematically.

## 4.2 `first_of()`

If

$$E = \texttt{first_of}(E_1, \ldots, E_m),$$

then the rule becomes true when the first child becomes true. Therefore

$$T_E = \min(T_{E_1}, \ldots, T_{E_m}),$$

and

$$F_E(t) = 1 - \prod_{r=1}^{m} S_{E_r}(t), \qquad f_E(t) = \sum_{r=1}^{m} f_{E_r}(t) \prod_{q \neq r} S_{E_q}(t). \tag{4}$$

The density has the usual interpretation: one child finishes at time $t$, and all the others are still unfinished at time $t$.

## 4.3 `all_of()`

If

$$E = \texttt{all_of}(E_1, \ldots, E_m),$$

then the rule becomes true when the last required child becomes true. Therefore

$$T_E = \max(T_{E_1}, \ldots, T_{E_m}),$$

and

$$F_E(t) = \prod_{r=1}^{m} F_{E_r}(t), \qquad f_E(t) = \sum_{r=1}^{m} f_{E_r}(t) \prod_{q \neq r} F_{E_q}(t). \tag{5}$$

Here one child finishes at time $t$, and all the others have already finished by time $t$.

### 4.4 `none_of()`

`none_of()` is an absence condition. If a rule requires `none_of(B)`, then the relevant term is simply the survival function of $B$:

$$\Pr(B \text{ has not finished by time } t) = S_B(t).$$

For example, if

$$E = \texttt{all_of}(A, \texttt{none_of}(B)),$$

then by time $t$ the rule holds with probability

$$F_E(t) = F_A(t) S_B(t).$$

So `none_of()` does not introduce a new race winner. It contributes a survival factor.

### 4.5 `inhibit()`

The blocking rule is written as

$$G = \texttt{inhibit}(A, B).$$

Its meaning is simple: count $A$ only if $B$ has not finished first. The corresponding density is

$$f_G(t) = f_A(t) S_B(t), \qquad F_G(t) = \int_0^t f_G(u)\, du. \tag{6}$$

So $A$ finishes at time $t$, while $B$ is still unfinished at time $t$.

## 5 Trial Likelihood

Suppose the model defines outcomes $R_1, \ldots, R_J$. If response $r$ is observed at time $t$, then the trial contribution is

$$\ell_r(t) = \frac{d}{dt} \Pr\big(T_{R_r} \leq t,\ T_{R_j} > t \text{ for all } j \neq r\big)\,. \tag{7}$$

This is the general "response $r$ wins at time $t$" formula.

### 5.1 Disjoint outcome rules

The simplest case is when the outcome rules are *disjoint*. Here "disjoint" means that the same latent trial cannot satisfy two different outcome rules at once. Most standard race models have this form: one response wins, and the others have not yet become true.

In that case, Equation (7) reduces to

$$\ell_r(t) = f_{R_r}(t) \prod_{j \neq r} S_{R_j}(t). \tag{8}$$

This is the familiar race-model likelihood: response $r$ finishes at time $t$, and every competing response is still unfinished at time $t$. This is the form most readers will already know from ordinary race models.

### 5.2 Outcome rules with shared structure

Sometimes different outcome rules share part of the same latent machinery. For example:

$$\texttt{left} = \texttt{all_of}(A, C), \qquad \texttt{right} = \texttt{all_of}(B, C).$$

Both responses depend on the common process $C$. Their completion times are

$$T_{\texttt{left}} = \max(T_A, T_C), \qquad T_{\texttt{right}} = \max(T_B, T_C).$$

If $C$ finishes before both $A$ and $B$, the model behaves like an ordinary race between $A$ and $B$. If $C$ finishes after both, then

$$T_{\texttt{left}} = T_{\texttt{right}} = T_C.$$

So shared structure can create positive-probability ties between composite outcome times, even when the underlying leaf times are continuous.

For that reason, Equation (8) is no longer enough. The competitor term can no longer be handled by a simple product of survival functions, because competing rules may share part of the same completion event.

A useful way to write the likelihood is to sum over the different ways outcome rule $r$ can become true:

$$\ell_r(t) = \sum_{m \in M_r} f_{r,m}(t)\, p^{\text{win}}_{r,m}(t). \tag{9}$$

Here $M_r$ is the set of completion modes for outcome $r$, $f_{r,m}(t)$ is the density that mode $m$ makes $r$ available at time $t$, and $p^{\text{win}}_{r,m}(t)$ is the probability that this mode wins against the competing rules.

The quantity $p^{\text{win}}_{r,m}(t)$ is easiest to describe through its complement,

$$1 - p^{\text{win}}_{r,m}(t) = \Pr(\text{at least one competing rule beats mode } m \text{ at time } t),$$

and this is a union probability. So it is evaluated by inclusion–exclusion:

$$1 - p^{\text{win}}_{r,m}(t) = \sum_{s} W_{rm,s}(t) - \sum_{s<u} W_{rm,su}(t) + \cdots, \tag{10}$$

where $W_{rm,s}(t)$ is the probability that competitor $s$ beats mode $m$, $W_{rm,su}(t)$ the probability that competitors $s$ and $u$ do so jointly, and so on.

In the running example, one completion mode for `left` is that $C$ finishes at time $t$ after $A$ is already ready. Then $f_{r,m}(t)$ is the density of that event. The competitor term asks whether the `right` rule can also claim that same shared finishing stage. If $B$ is not ready, it cannot. If $B$ is ready as well, then left and right share the same final time $T_C$, and the winner depends on which branch was ready first.

**No-response mass.** Some models leave positive probability on "no observed response", for example because a trigger fails. Before any relabelling or guessing, that probability is

$$p_{\text{none}} = 1 - \sum_{r=1}^{J} \int_0^\infty \ell_r(t)\, dt. \tag{11}$$

So the no-response mass is whatever probability remains after summing the finite response probabilities.

## 6 Onsets

Onset parameters control when a leaf is allowed to begin. They do not change the post-start distribution itself. They only change how that post-start distribution is placed in time.

**Absolute onset.** A leaf may have an absolute onset delay $d_a \geq 0$. If $T^\star_a$ is the baseline finishing time after the process has started, then

$$T_a = d_a + T^\star_a.$$

Equivalently,

$$F_a(t) = F^\star_a(t - d_a), \qquad f_a(t) = f^\star_a(t - d_a),$$

with both terms equal to 0 for $t \leq d_a$. So an absolute onset simply shifts the finishing-time distribution to the right.

**Chained onset.** A leaf may also start only after another source has finished. If leaf $a$ starts after source $U$, with an additional lag $\delta \geq 0$, then

$$T_a = T_U + \delta + T_a^\star,$$

where $T_a^\star$ is the leaf's own post-start finishing time.

So the total waiting time is: wait for $U$, then wait for the lag, then wait for $a$ itself. This gives the convolution

$$f_a(t) = \int_0^{t-\delta} f_U(s)\, f_a^\star(t-\delta-s)\, ds, \tag{12}$$

and similarly

$$F_a(t) = \int_0^{\infty} F_a^\star(t-\delta-s)\, f_U(s)\, ds. \tag{13}$$

# 7 Triggers

Triggers control whether a source starts at all. Independent triggers act on a single leaf. Shared triggers introduce a trial-level on/off mixture across a group of leaves.

**Independent trigger.** If leaf $a$ has start probability $1 - q_a$, then the finite-time distribution is simply scaled by that probability:

$$F_a(t) = (1-q_a)F_a^\dagger(t), \qquad f_a(t) = (1-q_a)f_a^\dagger(t), \tag{14}$$

where $F_a^\dagger$ and $f_a^\dagger$ are the channels the leaf would have had without trigger failure.

**Shared trigger.** If several leaves share one trigger with failure probability $q_g$, then they either all start together or all fail together. The trial likelihood is then a two-part mixture:

$$L = (1-q_g)L^{(\mathrm{on})} + q_g L^{(\mathrm{off})}. \tag{15}$$

This is the only point at which the framework introduces joint start dependence across several leaves.

# 8 Observation Layer

## 8.1 Mixtures

A component $c$ defines one variant of the model, with weight $\pi_c$. Mixtures can then enter the likelihood in two different ways.

**Explicit mixture.** If the component is observed on a trial, then the trial contribution is simply

$$L = L_c.$$

This is just a model with different trial types. Each trial is assigned to one known component, and the likelihood conditions on that assignment.

**Marginalized mixture.** If the component is not observed, then each trial is treated as arising from one of several latent process variants, and the likelihood averages over them:

$$L = \sum_c \pi_c L_c, \qquad \sum_c \pi_c = 1. \tag{16}$$

So an explicit mixture separates known trial types, whereas a marginalized mixture keeps the component latent and forms a within-trial mixture over possible process variants.

## 8.2 Outcome remapping and guessing

The outcome generated by the model need not be the same as the label that is observed. Let $M_{yr}$ be the probability that latent outcome $r$ is recorded as observed label $y$. Then

$$\ell_y(t) = \sum_r M_{yr}\, \ell_r(t). \tag{17}$$

So the observed density is a weighted sum of the latent outcome densities.

If the observed label is kept but the response time is discarded, then the contribution becomes the integrated mass

$$p_y = \int_0^\infty \ell_y(t)\, dt. \tag{18}$$

## 8.3 Ranked responses

The package can also use data in which the first $K$ responses are observed on each trial:

$$(r_1, t_1), \ldots, (r_K, t_K), \qquad 0 < t_1 < \cdots < t_K.$$

This part of the framework is narrower than the single-response case. The current ranked implementation is for direct, disjoint outcomes.

In that setting, the ranked likelihood is

$$L_{\text{rank}} = \left[ \prod_{m=1}^{K} f_{r_m}(t_m) \right] \left[ \prod_{j \notin \{r_1, \ldots, r_K\}} S_j(t_K) \right]. \tag{19}$$

Because the times are ordered, the sequence $t_1 < \cdots < t_K$ already encodes which response came first, second, and so on.